

10. Critical Analysis and Open Problems

The preceding sections have traced a compelling arc from classical evolutionary computation to LLM-guided discovery of reinforcement learning algorithms. The results are impressive on their face, but intellectual honesty demands that we interrogate the foundations, limitations, and unresolved tensions of this emerging paradigm. This section offers a systematic critical analysis organized around seven open problems, followed by a discussion of future directions that could address them.

10.1 The Novelty Question

The most fundamental question facing LLM-guided algorithm discovery is whether the outputs constitute genuine scientific contributions or sophisticated recombination. Hazra et al. [Hazra2025b] are admirably transparent on this point, acknowledging that “the discovered algorithms arise from novel recombinations of existing reinforcement-learning mechanisms rather than from fundamentally new update forms.” The evolved loss functions combine entropy regularization, clipped ratios, and advantage weighting in configurations not previously explored, but every individual component has antecedents in the RL literature.

This admission invites a sharper critique articulated by Ting [Ting2025] in *Nature Astronomy*, who argues that if a large language model can readily replicate the core intellectual contribution of a scientific work, that contribution may lack sufficient novelty. Ting proposes what amounts to an “LLM replicability test” for scientific merit, suggesting that peer reviewers should evaluate whether a contribution lies meaningfully outside the model’s training distribution. Under this criterion, loss functions assembled from known RL primitives by an LLM would fail to qualify as genuinely novel discoveries.

There is, however, a credible counter-argument rooted in the innovation studies literature. Schumpeter [Schumpeter1939] defined innovation as *neue Kombinationen* — new combinations of existing means of production. Fleming [Fleming2001] formalized this insight in his theory of recombinant search, demonstrating empirically that breakthrough inventions arise disproportionately from the combination of components not previously joined. Under this framework, the LLM functions as a combinatorial search engine operating over a space of algorithmic primitives, and the novelty lies in the specific combinations explored rather than in the primitives themselves. The tension between these perspectives remains unresolved and reflects a deeper philosophical disagreement about the nature of scientific contribution that predates the LLM era entirely.

10.2 The Compute Problem

Even if one accepts the scientific value of evolved algorithms, the practical barrier of computational cost is severe. In the Sygkounas et al. framework [Hazra2025b], each candidate algorithm requires 25 full training runs (5 environments multiplied by 5 random seeds) to obtain a reliable fitness estimate. With 24 new candidates generated per island per generation across 10 generations, the total evaluation budget runs into the thousands of complete RL training procedures. Each generation consumed approximately 30 hours of wall-clock time on four A100 GPUs with 40 GB of memory, and the work required access to the LUMI supercomputer through EuroHPC. FunSearch [RomeraParedes2024] and AlphaEvolve [Novikov2025] similarly depend on Google DeepMind-scale infrastructure, with AlphaEvolve orchestrating Gemini 2.0 Flash and Gemini 2.0 Pro models in tandem for candidate generation.

This creates an uncomfortable asymmetry in the research landscape. The methodology is, in principle, general-purpose, yet in practice it is accessible only to groups with supercomputer

allocations or industry backing. The broader RL research community — particularly at smaller universities and in developing countries — cannot replicate these experiments, let alone extend them. If the most promising direction in automated algorithm design requires resources available to fewer than a dozen institutions worldwide, the field risks becoming a spectator sport rather than a participatory science.

10.3 The LLM Dependence Problem

Perhaps the most alarming finding in Hazra et al. [Hazra2025b] is the dramatic sensitivity of results to the choice of language model. GPT-5.2 achieved final fitness scores in the range of 0.65 to 0.69, while Claude 4.5 Opus produced scores of only 0.37 to 0.47, a gap so large that the Claude-evolved algorithms were excluded from further analysis entirely. This is not a minor implementation detail. It means that the scientific conclusions of the paper — which algorithms were discovered, which mechanisms proved effective, which combinations were fruitful — are contingent on the specific proprietary model used as the evolutionary operator.

This dependence creates three distinct problems. First, reproducibility is undermined because API-vended models are updated, versioned, and occasionally deprecated without guarantees of behavioral stability. An experiment conducted with GPT-5.2 in March 2026 may not yield the same results with GPT-5.2 in March 2027, or with whatever model succeeds it. Second, the science becomes entangled with commercial interests, as results that “only work with OpenAI’s API” are not fully open science. Third, the sensitivity itself is poorly understood. Why does GPT-5.2 outperform Claude 4.5 Opus by such a wide margin on this task? Is it the code generation capability, the implicit knowledge of RL algorithms, or some artifact of the prompt format? Without answers to these questions, the LLM is a black box within a black box, and the evolved algorithms are doubly opaque.

10.4 The Generalization Problem

The evaluation protocol in Hazra et al. [Hazra2025b] trains evolved algorithms on five Gymnasium environments and tests on ten (five training plus five novel). The results reveal a characteristic pattern familiar from the meta-learning literature [Yao2021, Kirsch2020]: algorithms that are competitive on the training distribution do not reliably transfer. The best evolved algorithm, CG-FPD, achieved a score of 2,407.88 on HalfCheetah-v5, compared to SAC’s 4,988.5, and 1,603.80 on Walker2d-v5 versus SAC’s 4,595.30. These are not marginal shortfalls but substantial performance gaps on standard continuous control benchmarks. On the novel Swimmer environment, a different evolved algorithm (DF-CWP-CP) performed well, but it underperformed on Reacher and Walker2d.

This pattern echoes the broader challenge of meta-overfitting to task distributions that has been documented extensively in meta-reinforcement learning [Yao2021]. When algorithms are optimized for aggregate fitness across a fixed set of environments, the evolutionary pressure may exploit shared structure in those environments rather than discovering genuinely general learning principles. The analogy to overfitting in supervised learning is direct and troubling: just as a neural network can memorize training data, an evolutionary process can “memorize” a benchmark suite.

10.5 The Theory Vacuum

Classical reinforcement learning, for all its practical difficulties, rests on theoretical foundations. Q-learning converges to the optimal action-value function under well-defined conditions [Watkins1992]. Policy gradient methods have known convergence properties [Sutton1999]. Even

actor-critic algorithms possess finite-time convergence guarantees under function approximation [Konda2000]. None of these guarantees exist for any LLM-evolved algorithm. There are no convergence proofs, no sample complexity bounds, and no formal characterization of the search space over which evolution operates.

This absence is not merely an aesthetic concern. Without theoretical grounding, practitioners have no basis for predicting when an evolved algorithm will succeed or fail, no way to diagnose failures systematically, and no guarantee that training will not diverge on a new task. The contrast with the classical paradigm is stark: SAC may underperform an evolved algorithm on three benchmarks, but at least its maximum entropy objective is understood analytically, its soft Bellman equation is well-defined, and its convergence behavior under mild assumptions is characterized [Haarnoja2018]. The evolved algorithms offer performance without understanding, which is a fragile bargain.

10.6 The Reproducibility Crisis

Henderson et al. [Henderson2018] documented the reproducibility crisis in deep RL, demonstrating that implementation details, hyperparameter choices, random seeds, and even codebases can produce dramatically different results for nominally identical algorithms. The LLM-guided evolution paradigm layers two additional sources of stochasticity on top of these existing difficulties: the randomness of the evolutionary search process and the stochasticity inherent in LLM generation. The combination is formidable. Even if the LLM, the evolutionary parameters, the evaluation environments, and the random seeds are all held fixed, the temperature parameter in the LLM’s sampling introduces irreducible variation in the candidate programs generated.

Furthermore, the reporting conventions in this nascent field have not yet matured. When results are presented for the “best evolved algorithm” selected post hoc from a population of candidates across multiple evolutionary runs, the effective degrees of freedom in the analysis are enormous. This is analogous to the “best checkpoint” reporting that Henderson et al. [Henderson2018] criticized in the broader deep RL literature, amplified by the additional selection pressure of choosing from an entire population of evolved candidates.

10.7 The Evaluation Bottleneck

Irpan [Irpan2018] argued persuasively in 2018 that deep RL “doesn’t work yet,” cataloging the gap between impressive benchmark results and the requirements of real-world deployment. Eight years later, the situation has improved but the core critique retains force, particularly for LLM-evolved algorithms. The Gymnasium environments used for evaluation — CartPole, LunarLander, HalfCheetah, Walker2d — are the field’s standard benchmarks precisely because they are tractable and well-studied, but they are also stylized, low-dimensional, and far removed from production applications. An evolved algorithm that matches PPO on CartPole tells us very little about its behavior in robotics, autonomous driving, or large-scale recommendation systems.

The evaluation bottleneck is self-reinforcing. Because each candidate evaluation is expensive (as discussed in Section 10.2), researchers are incentivized to use the simplest environments that produce publishable results. This creates a feedback loop in which evolved algorithms are optimized for and validated on benchmarks that are insufficient to establish practical relevance, while the cost of more rigorous evaluation remains prohibitive.

10.8 Future Directions

Despite these criticisms, the paradigm is not without promise, and several research directions could address the limitations identified above. First, surrogate fitness models could dramatically reduce evaluation cost. The surrogate-assisted evolutionary algorithm literature [Jin2011] has demonstrated that learned approximations of the fitness function can reduce the number of expensive evaluations by orders of magnitude, and applying these techniques to algorithm evolution is a natural next step.

Second, the LLM dependence problem could be mitigated by the use of open-weight models. The OpenELM library [Lehman2024] has already demonstrated that evolutionary search with LLMs need not require proprietary APIs, and the rapid improvement of open models such as Llama and Mistral suggests that competitive performance may soon be achievable without commercial dependencies.

Third, the theory vacuum could be partially addressed through formal verification of evolved algorithms. If the search space is constrained to produce algorithms with particular structural properties — for instance, algorithms expressible as contractive operators — then convergence guarantees could potentially be inherited from the structural constraints rather than proved *de novo*.

Fourth, the evaluation bottleneck calls for the development of richer benchmark suites designed specifically for algorithm discovery. These should include environments with different reward structures, observation spaces, and dynamics, explicitly testing transfer rather than in-distribution performance.

Finally, the most transformative possibility is the integration of LLM self-improvement during the evolutionary search, as explored in recent co-evolutionary approaches [Surina2025]. If the LLM itself can be fine-tuned on successful evolutionary discoveries, the search process becomes an engine of self-improvement rather than a static exploitation of pre-trained capabilities. Whether this leads to a genuine positive feedback loop or merely to faster convergence to the same recombinatory ceiling remains an open and fascinating question.

References

- [Fleming2001] L. Fleming. “Recombinant Uncertainty in Technological Search.” *Management Science*, 47(1):117–132, 2001.
- [Haarnoja2018] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine. “Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor.” *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.
- [Hazra2025b] R. Hazra, A. Sygkounas, A. Persson, A. Loutfi, P. Zuidberg Dos Martires. “Evolutionary Discovery of Reinforcement Learning Algorithms via Large Language Models.” *arXiv preprint arXiv:2603.28416*, 2025.
- [Henderson2018] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, D. Meger. “Deep Reinforcement Learning that Matters.” *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), 2018.
- [Irpan2018] A. Irpan. “Deep Reinforcement Learning Doesn’t Work Yet.” *Blog post*, February 2018. Available at: <https://www.alexirpan.com/2018/02/14/rl-hard.html>.
- [Jin2011] Y. Jin. “Surrogate-Assisted Evolutionary Computation: Recent Advances and Future Challenges.” *Swarm and Evolutionary Computation*, 1(2):61–70, 2011.

- [Kirsch2020] L. Kirsch, S. van Steenkiste, J. Schmidhuber. “Improving Generalization in Meta Reinforcement Learning using Learned Objectives.” *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [Konda2000] V. R. Konda and J. N. Tsitsiklis. “Actor-Critic Algorithms.” *Advances in Neural Information Processing Systems (NeurIPS)*, 2000.
- [Lehman2024] J. Lehman et al. “Evolution through Large Models.” *Genetic Programming and Evolvable Machines*, 25:2, 2024. OpenELM Library: <https://github.com/CarperAI/OpenELM>.
- [Novikov2025] A. Novikov, M. Balog, N. Vu et al. “AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery.” *arXiv preprint arXiv:2506.13131*, 2025.
- [RomeraParedes2024] B. Romera-Paredes, M. Barekatin, A. Novikov et al. “Mathematical Discoveries from Program Search with Large Language Models.” *Nature*, 625:468–475, 2024.
- [Schumpeter1939] J. A. Schumpeter. *Business Cycles: A Theoretical, Historical, and Statistical Analysis of the Capitalist Process*. McGraw-Hill, 1939.
- [Surina2025] A. Surina, N. Mansouri et al. “Algorithm Discovery with LLMs: Evolutionary Search Meets Reinforcement Learning.” *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.
- [Sutton1999] R. S. Sutton, D. McAllester, S. Singh, Y. Mansour. “Policy Gradient Methods for Reinforcement Learning with Function Approximation.” *Advances in Neural Information Processing Systems (NeurIPS)*, 1999.
- [Ting2025] Y.-S. Ting. “Artificial Intelligence Compels the Astronomy Community to Rethink Research Identity and Redefine Excellence.” *Nature Astronomy*, 9:317–318, 2025.
- [Watkins1992] C. J. C. H. Watkins and P. Dayan. “Q-Learning.” *Machine Learning*, 8(3):279–292, 1992.
- [Yao2021] H. Yao, L. Huang, L. Zhang, Y. Wei, L. Tian, J. Zou. “Improving Generalization in Meta-Learning via Task Augmentation.” *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.